SQL Explanation Grader: baseline vs RAG

Experiment **rag_v4_dev** · split **dev** (50 items) · generator **llama3.2:3b** · judge **qwen2.5:3b** (legacy_schema) · generated 2026-10-26 18:08

How to read this report. "Judge approval" is the share of explanations an LLM judge graded good. It is an automated, unverified measurement, not ground truth. Deterministic checks are rule-based heuristics computed from the SQL structure. Agreement of the judge with reference labels is reported separately (results/rag/judge_agreement.json).

Headline comparison

Metric	Baseline (prompt only)	RAG
Judge approval (95% CI)	30.0% (15/50) [19-44]	22.0% (11/50) [13-35]
Approval, abstentions count as fail	30.0% (15/50) [19-44]	22.0% (11/50) [13-35]
Rubric pass (judge's own scores)	n/a	n/a
Mean correctness (1-5)	n/a	n/a
Mean completeness (1-5)	n/a	n/a
Mean clarity (1-5)	n/a	n/a
Mean groundedness (1-5)	n/a	n/a
Explanations with unsupported claims (judge)	n/a	n/a
Deterministic check coverage	0.96	0.97
Invented numbers (rule-based)	10.0% (5/50) [4-21]	10.0% (5/50) [4-21]
Raw SQL keywords in text	10.0% (5/50) [4-21]	60.0% (30/50) [46-72]
Abstentions	0.0% (0/50) [0-7]	0.0% (0/50) [0-7]
Generation errors	0	0
Mean end-to-end latency (s)	6.6	7.7
Mean retrieval time (s)	0.00	0.63
Mean input tokens	571	1469
Mean output tokens	104	191

AI-assisted reference labels (not human): share labelled good

	Baseline	RAG
Labelled good	20/50	16/50
Paired: only baseline good / only RAG good	14	10
RAG minus baseline (95% bootstrap CI)	-8.0 pts (-26.0 to +12.0)	McNemar p = 0.541

Labels and their decision rules: data/labels/. They are not produced by the judge in this report.

Paired comparison (same items, both judged)

50 pairs. Both good: 4; both bad: 28; only baseline good: 11; only RAG good: 7. Approval difference RAG minus baseline: -8.0 points (95% bootstrap CI -24.0 to +8.0). Exact McNemar p = 0.4807.

Grounding and retrieval (RAG condition)

Metric	Value
Answers with at least one citation	64.0% (32/50) [50-76]
Citation validity (cited source was provided)	1.000
Answers citing a source that was not provided	0.0% (0/50) [0-7]
Share of sentences carrying a citation	0.312

Metric	Value
Lexical support of cited sentences (heuristic)	0.692
Model said evidence was insufficient	0
Retrieval feature coverage	1.000
Evidence judged sufficient before generation	100.0% (50/50) [93-100]
Grounding status counts	{"weakly_grounded": 18, "grounded": 32}

Retrieval evaluation (no LLM involved)

Project-authored by Claude (AI-assisted) on 2026-10-26 by reading the section headings of each corpus document. Not human-reviewed yet. Treat retrieval scores computed with it as rule-based estimates, not human relevance judgements.

Chunk	Strategy	Mode	MRR	Recall@5	Prec@5	nDCG@5	Ctx recall	ms/query
400	decomposed	dense	0.824	0.925	0.556	0.616	0.892	161
400	decomposed	bm25	0.642	0.836	0.568	0.560	0.960	5
400	decomposed	hybrid	0.690	0.867	0.585	0.590	0.893	162
400	probe	hybrid	0.564	0.596	0.397	0.417	0.711	50
400	raw_sql	hybrid	0.691	0.797	0.522	0.538	0.799	58
200	decomposed	dense	0.784	0.899	0.571	0.584	0.943	161
200	decomposed	bm25	0.710	0.837	0.637	0.627	0.963	7
200	decomposed	hybrid	0.640	0.878	0.604	0.569	0.948	163
300	decomposed	dense	0.764	0.676	0.511	0.548	0.934	162
300	decomposed	bm25	0.658	0.842	0.597	0.576	0.962	5
300	decomposed	hybrid	0.644	0.834	0.535	0.531	0.940	160
500	decomposed	dense	0.694	0.727	0.448	0.500	0.873	160
500	decomposed	bm25	0.728	0.839	0.589	0.604	0.968	4
500	decomposed	hybrid	0.647	0.854	0.566	0.564	0.874	166
600	decomposed	dense	0.743	0.645	0.458	0.532	0.932	162
600	decomposed	bm25	0.705	0.835	0.548	0.571	0.913	6
600	decomposed	hybrid	0.615	0.830	0.490	0.502	0.899	160

Breakdown by SQL feature (dataset label)

Feature	n	Baseline approval	RAG approval	Base checks	RAG checks
GROUP BY with HAVING	8	25.0% (2/8) [7-59]	12.5% (1/8) [2-47]	0.88	0.96
INNER JOIN of two tables	9	66.7% (6/9) [35-88]	22.2% (2/9) [6-55]	1.00	1.00
JOIN of three tables with ORDER BY and LIMIT	8	37.5% (3/8) [14-69]	37.5% (3/8) [14-69]	0.88	0.88
LEFT JOIN with IS NULL filter	3	100.0% (3/3) [44-100]	66.7% (2/3) [21-94]	1.00	1.00
simple SELECT with WHERE	5	0.0% (0/5) [0-43]	0.0% (0/5) [0-43]	1.00	1.00
subquery in WHERE	10	10.0% (1/10) [2-40]	30.0% (3/10) [11-60]	1.00	1.00
window function such as ROW_NUMBER or RANK	7	0.0% (0/7) [0-35]	0.0% (0/7) [0-35]	1.00	1.00

Items where the two conditions were graded differently

q029 · INNER JOIN of two tables

```
SELECT patients.id, doctors.id, appointments.appt_date, patients.name AS patient_name, doctors.name AS doctor_name FROM patients INNER JOIN appointments ON patients.id = appointments.patient_id INNER JOIN doctors ON appointments.doctor_id = doctors.id;
```

Baseline: good
This query retrieves a list of patients and their corresponding appointments, along with the doctor's name and appointment date. The query starts by selecting the patient's ID and the doctor's ID, as well as the appointment date. It then joins the patients table with the appointments table on the patient's ID, and the appointments table with the doctors table on the doctor's ID. The result is a list of patients with their corresponding appointments, including the doctor's name and appointment date. The query does not include any filters or grouping, and it returns all rows from the joined tables. The result set includes the patient's ID, the doctor's ID, the appointment date, the patient's name, and the doctor's name, with the patient's name and doctor's name aliased as patient_name and doctor_name, respectively.
Judge: The explanation correctly describes the query's purpose, joins, and columns selected. It also clarifies the aliasing used.

RAG: bad
This query returns the patient's ID, doctor's ID, appointment date, patient's name, and doctor's name. It joins the patients table with the appointments table on the patient's ID, and then joins the appointments table with the doctors table on the doctor's ID. [S1] The query uses an INNER JOIN, which returns only the rows where the join condition is met, and is equivalent to using the comma operator or the CROSS JOIN operator in this case. [S1] The CROSS JOIN operator is not typically used in this situation, as it would return all possible combinations of patients and doctors, which is not the desired result. [S1] Instead, the query uses the INNER JOIN operator, which is a more common and efficient way to join tables in SQLite. [S1] The query does not use any subqueries, aggregations, or window functions, and simply returns the specified columns from the joined tables.
Judge: It incorrectly states that the query uses a CROSS JOIN, which is not the case. The query uses an INNER JOIN.

---

q032 · INNER JOIN of two tables

```
SELECT rooms.price_per_night FROM rooms INNER JOIN bookings ON rooms.id = bookings.room_id;
```

Baseline: good
This query returns the price per night for each room type. It does this by matching rows from the rooms table with rows from the bookings table where the room ID in the rooms table matches the room ID in the bookings table. The result is a list of all room types and their corresponding price per night, without any additional information about the guests or bookings. The query does not include any specific room types or price ranges, and it does not limit the number of rows returned. The result is a simple list of room types and their prices, without any additional details.
Judge: The explanation correctly describes the query's operation, covers all aspects of the query, and uses clear language.

RAG: bad
This query returns the price per night for each room, along with the rooms that have been booked for a certain number of nights. The query joins the rooms table with the bookings table on the room_id column, which is the ID of the room that the guest is booked into. [S1] The join is an INNER JOIN, which means that only rows with matching room_id values in both tables are included in the result. This is equivalent to using the JOIN operator or the comma operator, and is handled differently by the query optimizer in SQLite. [S1] The CROSS JOIN operator would produce the same result, but is generally not recommended for use in this situation, as it can influence the query optimizer's choice of algorithm and is not part of standard SQL. [S1]
Judge: It incorrectly states that the query returns the price per night for each room, along with the rooms that have been booked for a certain number of nights. The query only returns the price per night and does not include the booking information.

---

q034 · INNER JOIN of two tables

```
SELECT rooms.room_type FROM rooms INNER JOIN bookings ON rooms.id = bookings.room_id;
```

Baseline: bad
This query returns the types of rooms that are currently booked. It joins the rooms table with the bookings table on the id column, which matches the room id in the bookings table. The result is a list of room types that have been booked, without any information about the guests or the price of the rooms. The query does not include any rooms that are not currently booked, because it uses an INNER JOIN, which only includes rows where the matching columns in both tables have the same value. The result is a single calculated value, which is a list of room types, rather than a count of rows.
Judge: The explanation incorrectly states that the query does not include any rooms that are not currently booked. An INNER JOIN does not exclude non-booked rooms, it simply does not include them in the result if there is no match in the other table. The query does not include a count of rows, it returns a list of room types.

RAG: good
This query returns the room types from the rooms table, joined with the bookings table on the room_id column, which means it only includes bookings for rooms that have been booked.
Judge: The explanation correctly describes the query's operation, including the use of INNER JOIN and the filtering effect on the bookings table.

---

q043 · subquery in WHERE

```
SELECT * FROM loans WHERE book_id IN (SELECT id FROM books WHERE year > 2000);
```

Baseline: good
This query retrieves all rows from the loans table where the book_id matches a book that was published after 2000. In other words, it shows all loans of books published in the 21st century. The subquery first identifies the book_ids of books published in the 21st century, and then the main query selects the corresponding loan records.
Judge: The explanation correctly describes the query's operation, covers all parts of the query, and uses clear language.

RAG: bad
This query selects all rows from the loans table where the book_id is in the list of book_ids from the books table that were published after 2000. In other words, it finds all loans of books published in 2000 or later. The subquery uses the IN operator to compare the book_id in each loan row to the list of book_ids from the books table, and the ALL operator is used to ensure that all book_ids in the list must match the book_id in each loan row.
Judge: The explanation incorrectly states that the ALL operator is used, but the subquery uses IN without specifying ALL. The query structure is correct but the explanation is misleading.

---

q048 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT clients.name, accounts.type, transactions.tx_date FROM clients JOIN accounts JOIN transactions ON clients.id = accounts.client_id AND accounts.id = transactions.account_id ORDER BY clients.id LIMIT 1;
```

Baseline: bad
This SQL query retrieves the name of a client, the type of their account, and the date of the most recent transaction. The query starts by joining three tables: clients, accounts, and transactions. It then filters the results to only include rows where the client's ID matches the account's client ID and the account's ID matches the transaction's account ID. The results are ordered by the client's ID, so the most recent transaction for each client is first. Finally, the query limits the results to just one row per client, effectively showing the most recent transaction for each client.
Judge: The query does not select the most recent transaction for each client. It should use a subquery or a window function to find the most recent transaction date for each client. The current query only orders by client ID, not by transaction date.

RAG: good
This query returns the client's name, account type, and transaction date for the first account of the first client. The results are ordered by client ID in ascending order, and only the first row is returned.
Judge: The explanation correctly describes the query's operation, including the joins, filters, grouping, aggregate, ordering, and LIMIT. It is clear and non-technical.

---

q068 · LEFT JOIN with IS NULL filter

```
SELECT * FROM members LEFT JOIN loans ON members.id = loans.member_id WHERE loans.book_id IS NULL;
```

Baseline: good
This query retrieves all members who have never borrowed a book. It starts by selecting all columns from the members table. Then, it joins the members table with the loans table on the condition that the member's ID matches the loan's member ID. The LEFT JOIN ensures that all members are included, even if they don't have a matching loan record. The WHERE clause filters the results to only include loans where the book ID is NULL, meaning the member has not borrowed a book. This effectively selects all members who have never borrowed a book, along with any additional information from the members table.
Judge: The explanation covers every aspect of the query, including the join, filter, and the fact that it returns all members who have never borrowed a book.

RAG: bad
This query returns all members who have never borrowed a book, along with the book they would have borrowed if they had. The query joins the members table with the loans table on the condition that the member has borrowed a book (i.e., loans.book_id is not null), and returns all members who do not meet this condition. [S1] The query uses a LEFT JOIN to include all members, even if they have not borrowed a book, and adds NULL values to the columns that would normally contain book information. [S1] The join operations are processed in order from left to right, so the query first joins the members table with the loans table, and then adds the book information for the members who have borrowed a book. [S1] Extra rows of output are added for each member who has not borrowed a book, and for each book that has been borrowed by a member. [S1] Unmatched columns are filled in with NULL values.
Judge: The explanation incorrectly states that the query returns all members who have never borrowed a book, along with the book they would have borrowed if they had. The query actually returns members who have not borrowed a book, but it does not include the book information. The query uses a LEFT JOIN to include all members, even if they have not borrowed a book, and adds NULL values to the columns that would normally contain book information. However, the explanation does not correctly describe the LEFT JOIN and the resulting NULL values. It also incorrectly states that the query includes the book information for members who have borrowed a book.

---

q071 · subquery in WHERE
```
SELECT name FROM patients WHERE id = (SELECT patient_id FROM appointments WHERE id = 1);
```

Baseline: bad
This query finds the name of a patient who has an appointment with a specific doctor. The query first finds the patient ID of the patient who has the first appointment, and then uses that ID to find the name of the patient in the patients table. The result is the name of the patient who has the first appointment with a specific doctor.
Judge: The explanation does not cover the full query. It misses the part about the doctor's specialty and the filtering of appointments by doctor_id.

RAG: good
This query selects the name of a patient from the patients table where the patient's id matches the patient_id of the appointment with id 1.
Judge: The explanation correctly identifies the query's operation, covers the join and filter, and explains the use of subquery correctly.

---

q082 · subquery in WHERE
```
SELECT students.name, courses.title FROM students JOIN enrollments ON students.id =
enrollments.student_id JOIN courses ON enrollments.course_id = courses.id WHERE courses.credits =
(SELECT MAX(credits) FROM courses);
```

Baseline: bad
This query selects the names of students and the titles of courses they are enrolled in. It starts by joining the students table with the enrollments table, where each student is matched with the courses they are enrolled in, and then joins the enrollments table with the courses table. The query then filters the results to only include courses with the highest number of credits. The result is a list of students and the courses they are enrolled in, with only the courses having the most credits.
Judge: The explanation does not cover the subquery used to filter courses by the maximum number of credits.

RAG: good
This query returns the names of students enrolled in the course with the most credits. It joins the students, enrollments, and courses tables based on their IDs, and then filters the results to only include the course with the highest number of credits.
Judge: The explanation correctly describes the query's operation, covers all aspects of the query (joins, filters, grouping, and ordering), and uses clear language.

---

q083 · subquery in WHERE
```
SELECT students.name, courses.title FROM students JOIN enrollments ON students.id =
enrollments.student_id JOIN courses ON enrollments.course_id = courses.id WHERE courses.credits >
(SELECT AVG(credits) FROM courses);
```

Baseline: bad
This query selects the names of students and the titles of courses they are enrolled in. It joins the students, enrollments, and courses tables to find the courses each student is taking. The query then filters the results to only include courses that have more credits than the average credits of all courses. In other words, it shows students who are taking courses that are significantly more challenging than the average course.
Judge: The explanation does not cover the WHERE clause correctly. It should state that the query filters courses by having more credits than the average, not that it shows students taking significantly more challenging courses.

RAG: good
This query returns the names of students who are enrolled in courses with more credits than the average credits of all courses. It does this by joining the students, enrollments, and courses tables, filtering the results to only include courses with more credits than the average, and then selecting the names of the students enrolled in those courses. [S1]
Judge: The explanation correctly describes the query's operation, covers all aspects of the query (joins, filters, grouping, aggregate, ordering), and uses clear language.

---

q089 · GROUP BY with HAVING

```
SELECT course_id, COUNT(*) FROM enrollments GROUP BY course_id HAVING COUNT(*) % 2 = 0;
```

Baseline: bad
This query selects the course IDs of students who have enrolled in an even number of courses. It does this by counting the number of courses each student is enrolled in, grouping the results by course ID, and then filtering the groups to only include those with an even count of courses. The exact course IDs returned are the ones that meet this condition, which is why the query uses the `COUNT(*)` function to count the number of courses each student is enrolled in.
Judge: The explanation does not cover the GROUP BY and HAVING clauses, which are necessary for the query to work as intended.

RAG: good
This query returns the course_id of courses that have an even number of enrollments. It does this by counting the number of enrollments for each course using the COUNT(*) function, grouping the results by course_id, and then filtering the groups to only include those with an even count. [S1] The HAVING expression is evaluated across all rows in the group, and in this case, it's checking if the count is even. [S2] The COUNT(*) function returns the total number of rows in the group, which is then used to calculate the count of enrollments for each course.
Judge: The explanation correctly describes the query's operation, covers all aspects of the query (join, filter, grouping, aggregate, and HAVING), and uses clear language.

---

q096 · INNER JOIN of two tables

```
SELECT orders.id, customers.name, products.name, orders.quantity, orders.order_date FROM orders INNER
JOIN customers ON orders.customer_id = customers.id INNER JOIN products ON orders.product_id =
products.id;
```

Baseline: good
This query retrieves information from three tables: customers, products, and orders. It selects specific columns from each table: the order ID, customer name, product name, order quantity, and order date. The query joins the orders table with the customers table on the customer ID, and then joins the result with the products table on the product ID. The result set includes all rows from the orders table, along with the matching customer and product information. The query does not filter or limit the number of rows returned, so it will include all orders, customers, and products that are associated with each order.
Judge: The explanation covers every aspect of the query, including the joins, selected columns, and the structure of the query. It is clear and non-technical.

RAG: bad
This query returns the customer's name, product name, order quantity, and order date for each order. It joins the orders table with the customers table on the customer_id column and the products table on the product_id column, using an INNER JOIN to only include orders where the customer and product exist. [S1] The INNER JOIN, JOIN, and comma operators are interchangeable in SQLite, producing the same result as a CROSS JOIN, but with different handling by the query optimizer.
Judge: Explains INNER JOIN as interchangeable with CROSS JOIN, which is incorrect and not covered by the query.

---

q097 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT patients.name, doctors.name, appointments.appt_date FROM patients JOIN appointments ON
patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY
appointments.appt_date LIMIT 1;
```

Baseline: good
This SQL query retrieves the names of patients, doctors, and appointment dates from the database. It starts by joining the patients, appointments, and doctors tables based on their matching IDs. The query then orders the results by the appointment date in ascending order, so the earliest appointment date appears first. Finally, it limits the results to a single row, effectively showing the earliest appointment date and the corresponding patient and doctor names.
Judge: The explanation correctly describes the query's operations, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

RAG: bad
This query returns the names of patients, doctors, and the date of their appointment, sorted by appointment date, and limited to one row. The query joins the patients, appointments, and doctors tables based on their IDs, and then orders the results by the appointment date. The LIMIT clause is used to limit the number of rows returned to one, which means the query will only return the first row of the sorted result set. [S1] The join operators used in this query are interchangeable, and the CROSS JOIN operator is not used here. [S2] The ORDER BY clause is applied to the last simple SELECT in the compound query, which is the SELECT of appointments, and it sorts the rows based on the appt_date column. The rows are first sorted by appt_date, and then ties are broken by the next expression in the ORDER BY list, which is not present here. [S3] The LIMIT clause is applied to the last simple SELECT in the compound query, which is the SELECT of appointments, and it limits the number of rows returned to one. The LIMIT expression evaluates to an integer, which is 1, and the query returns the first row of the sorted result set.
Judge: The explanation does not cover every aspect of the query. It misses explaining the join conditions and the LIMIT clause's effect on the result set.